3. Introduction to Data Tables

Data Science with Applications in Natural Language Processing

Learning goals

By the end of this chapter we will be able to:

- **turn** collections of texts into data tables
- **read** data from Excel, CSV and URLs
- **explain** how DataFrame, Series and index relate
- **add** features derived from the text
- **choose** a storage format: CSV, compression, pickle

Chapter map

1. From raw text to a data table
2. The data table as a source of analysis
3. Efficient data storage

Three real datasets from the humanities: Homeric verses, papyri, papyrus images.

A text never comes alone

A list of texts

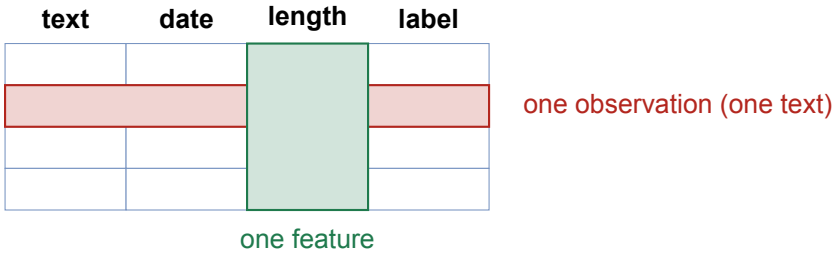
["I support the reform",
"The new policy is unfair"]

A data table

text	author	date	reply_to
I support the...	u17	2024-03-02	—
The new policy...	u42	2024-03-02	u17

- Who wrote it? When? Where? Replying to whom?
- **Metadata** need structure

Row = observation, column = feature



Three real datasets

Dataset	Content	Format	Question
HoLM	12,107 verses of the Odyssey	Excel	interpolations in the text
PaDoC	389 papyrus transcriptions	CSV	dating
PaLiT	images of literary papyri	Excel + images	dating from images

All public on GitHub, all from published studies.

HoLM: “linguistic surprise” in Homer

- The authorship of the Homeric epics has been debated for centuries
- Suspected **interpolations** concern individual verses
- Each verse gets a “surprise” score from a language model
- Correlated with named entities, n-grams, word frequency

Pavlopoulos et al., LREC-COLING 2024.

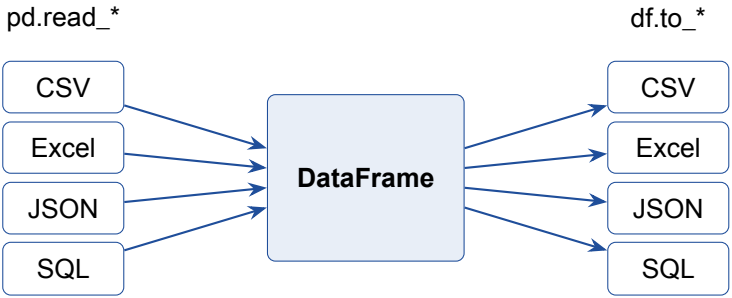
Reading from Excel

```
import pandas as pd

url = ("https://github.com/ipavlopoulos/holm/raw/"
       "refs/heads/main/holm_odyssey.xlsx")
holm = pd.read_excel(url, index_col=0)
holm.shape     # (12107, 10)
holm.sample()
```

- The suffix read_excel names the file format
- index_col=0: the first column becomes each row's key
- Without it: a redundant Unnamed: 0 column

One family of functions



The same logic for every format: only the suffix changes.

What does each column hold?

```
print(holm.columns.values)
# ['text' 'vid' 'bid' 'uid' 'ppl' 'ivf' 'ivf_avg'
#  'iliad_ppl' 'odyssey_ppl' 'cross_score']
```

text	the verse
vid, bid	verse number, book number
ppl	linguistic “surprise” (perplexity)
ivf	inverse frequency of the words across verses

Each column is a parallel representation of the same verse.

The index is a key, not a position

```
holm.index[:5]     # [5437, 9431, 5532, 2589, 6210]

holm.loc[5437]     # the row whose key is 5437
holm.iloc[0]       # the first row, whatever its key
```

- Here the verses are **shuffled**: the first row has key 5437
- loc: by key · iloc: by position
- That is why we set the index explicitly: for **control**

New features from the text

```
holm["char_length"] = holm["text"].str.len()
holm["word_count"]  = holm["text"].str.split().str.len()
holm[["text", "char_length", "word_count"]].head(3)
```

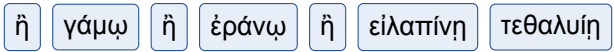
	text	char_length	word_count
5437	ἀνθῆσαι πυκάσαι τε γένυς ἐυανθέι λάχνῃ	38	6
9431	δωριέες τε τριχάϊκες δῖοί τε πελασγοί	37	6
5532	ἢ γάμῳ ἢ ἐράνῳ ἢ εἰλαπίνῃ τεθαλυίῃ	34	7

The .str accessor applies string methods to a whole column.

What is a token?

A unit of language

the pieces we cut a text into (tokenisation)



Here tokens = words split on spaces. They can also be subwords or characters.

The hexameter shows in the data

	characters	words
mean	42.3	7.2
standard deviation	3.2	1.3
minimum	30	3
maximum	56	14

- A very **narrow range**: the metre constrains length
- A simple describe() confirms what philology already knows

Discussion: what would you suspect about a verse with 14 words?

Pitfall: the mean of ppl

	ppl
mean	236,780
median	6,068
maximum	163,755,262

- A few extreme verses “pull” the mean
- Here the **median** describes a typical verse better
- For such distributions, a log scale helps

The extreme verses may be exactly what the study is looking for.

PaDoC: dating papyri

- 389 transcriptions of Greek documentary papyri
- From the 3rd century BC to the 7th century AD
- Many papyri can only be dated approximately
- Text regression models estimated the date of 159 manuscripts

Pavlopoulos et al., ACL 2023.

Reading from CSV

```
padoc = pd.read_csv("https://github.com/ipavlopoulos/padoc/"
                    "raw/refs/heads/main/data/padoc389.csv")
padoc.shape          # (389, 7)
padoc.columns
# ['tm', 'date', 'text_raw', 'url_img', 'url_trn',
#  'text_spaced', 'text']
```

- One row = **one text**
- tm: key · date: date · url_*: image and XML of the transcription

What does the date column mean?

	date
minimum	-2.5
median	1.8
maximum	6.8

- **Century** as a decimal number
- Negative values = BC (100 texts)
- The decimal part = position within the century

Before any analysis: what is the unit of each column?

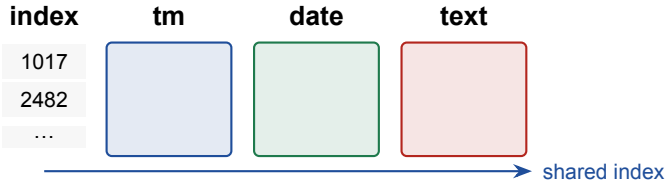
Pitfall: text without spaces

```
padoc["text"].iloc[0][:40]
# ετουσιπανημουεμβολιμουαφεσεκκαταπλουενβα''
padoc["text_spaced"].iloc[0][:40]
# ετουσ' ι πανημου εμβολιμου αφεσ εκ καταπλ'

padoc["text"].str.split().str.len().mean()   # 1.0
```

- Papyri were written **without spaces** (scriptio continua)
- In text, “word count” = 1 for every text
- Count words in text_spaced, characters in text

DataFrame = a collection of Series



- Each column = one **Series** with its own type
- All share the same **index**

Each Series has its own type

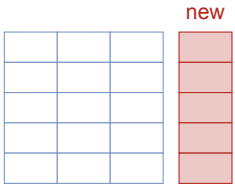
```
padoc.dtypes
# tm       int64
# date      float64
# text     object (string in pandas 3)
```

- Numeric columns: mean, describe, histograms
- Text columns: the .str accessor
- Each Series can be analysed **on its own**, without leaving the table

The table can grow

```
padoc["text_length"] = (
    padoc["text"].str.len())
```

- New column = new feature
- Same index as all the others



The index keeps columns in sync

```
df = pd.DataFrame({"text": ["α β", "γ", "δ ε ζ"]}, index=[10, 20, 30])
s = df["text"].str.len().sort_values()   # new order, same index

df["len_series"] = s        # aligned by index
df["len_list"] = list(s)    # aligned by position!
```

	text	len_series	len_list
10	α β	3	1
20	γ	1	3
30	δ ε ζ	5	5

A Series lands in the right row; a list does not always.

How long are the texts?

```
text_length = padoc["text"].str.len()
text_length.describe()
```

count	389
mean	824
std	722
min	38
25%	386
50%	643
75%	1,071
max	7,819

What the numbers tell us

- Mean length **824** characters: long transcriptions
- Standard deviation 722: **high variation**
- 38 characters: perhaps a fragmentary or damaged text
- Half the texts are up to 643 characters, three quarters up to 1,071
- Mean > median: a few very long texts

Same method, different scale

	HoLM (verse)	PaDoC (papyrus)
mean length	42 characters	824 characters
standard deviation	3	722

- The verse is **uniform**, the papyrus is not
- That affects later choices: normalisation, models, comparison

Often a CSV is enough

```
padoc.to_csv("padoc_clean.csv", index=False)
```

- Simple, readable by any tool
- Enough when we only have text and numbers

What if the data include images?

PaLiT: literary papyri

- Literary papyri: usually **undated**
- Palaeographic dating: often a range of more than a century
- 242 images of papyri in “bookhand”, with secure dates
- A network on images of text lines, with an error of about half a century

Pavlopoulos et al., Machine Learning, 2024.

Step 1: metadata from Excel

```
palit = pd.read_excel(
    "https://raw.githubusercontent.com/ipavlopoulos/palit/"
    "refs/heads/main/data/PaLit.xlsx",
    index_col=0,
    usecols=["uid", "date", "url", "filename_enc"])
palit.shape     # (255, 3)
```

- usecols: read only what we need
- uid becomes the index, not a column
- Still only structured information: dates, URLs, file names

Step 2: cleaning

```
noise = {18, 64, 66, 86, 98, 135, 140, 156}
palit = palit[~palit.index.isin(noise)]       # 247 records
palit = palit.drop_duplicates("filename_enc")   # 242 records
```

- Remove known problematic records
- Remove duplicate files
- Since uid is the index: palit.index, not palit.uid

Save versions, not just the final result



- We do not repeat expensive or fragile steps (downloads)
- Every experiment uses **exactly the same** data

Compression with no change in use

```
palit.to_csv("palit_metadata.csv.gz", index=False,
             compression="gzip")

pd.read_csv("palit_metadata.csv.gz")   # compression is detected
```

- Large corpora and images → compression matters
- Same code, smaller file

Step 3: downloading the images

```
import os, requests

IMG_DIR = "palit/data/PLF"
os.makedirs(IMG_DIR, exist_ok=True)
for uid, row in palit.iterrows():
    try:
        img = requests.get(row.url, timeout=30).content
        with open(f"{IMG_DIR}/{row.filename_enc}", "wb") as f:
            f.write(img)
    except Exception:
        print(f"ERROR: {row.url}")
```

One fixed IMG_DIR for download and loading; except Exception, not a bare except.

Keep only what was downloaded

```
downloaded = os.listdir(IMG_DIR)
palit = palit[palit["filename_enc"].isin(downloaded)]
len(palit)    # 173 in the book's run
```

- From 242 to 173: external resources **fail**
- The final dataset depends on when we ran the code
- One more reason to save the version we actually used

Step 4: images inside the table

```
from PIL import Image
import numpy as np

def load_image(path):
    try:
        return np.array(Image.open(path))
    except Exception:
        return None

palit["img_raw"] = palit["filename_enc"].apply(
    lambda name: load_image(f"{IMG_DIR}/{name}"))
```

Each cell of the new column is a NumPy array: not text, not a number.

Why not CSV here?

A CSV stores text and numbers, not NumPy arrays inside cells

- The image would become a huge string
- Its shape and type would be lost

Pickle: save the object as it is

```
palit[palit["img_raw"].notna()].to_pickle("palit.pkl")

palit = pd.read_pickle("palit.pkl")
```

- Serialises almost any Python object
- Metadata, annotations and images together
- Reloaded with **one command**, in exactly the same form

Careful with pickle

Compatibility

- tied to the Python and library versions
- a newer protocol → may not load in older Python

Security

- loading a pickle can **execute code**
- never read_pickle a file from an unknown source

For sharing and long-term storage: CSV, JSON, XML or NumPy files.

Which storage format?

Format	When	Limitation
CSV	text and numbers, sharing	basic types only
CSV.gz	the same, for large files	slightly slower to write
JSON	semi-structured, nested data	larger files
NumPy	numerical arrays, images	no column names
pickle	intermediate results, complex objects	fragile, not for sharing

Summary

1. A text comes with metadata: it needs a table
2. DataFrame = Series that share one index
3. The index is a key, not a position (loc vs iloc)
4. Before any measurement: what does each column mean?
5. Save versions, in the right format for each purpose

Discussion questions

1. Find the 5 verses with the highest ppl. Do they differ in length from the rest?
2. The download yielded 173 of 242 images. If the failures cluster in one century, what happens to the dating model?
3. When would you store a dataset as a pickle, and when not?